

Lecture 01: Introduction to Java

Introduction to Programming with Java

Agenda

1. What is Java?
2. A short history
3. Why learn Java?
4. How Java runs: source $\rightarrow$ bytecode $\rightarrow$ JVM
5. Where Java is used
6. How this course approaches Java
7. Looking ahead

What is Java?

Java is a general-purpose, object-oriented programming language.

- Designed for clarity, safety, and portability
- Programs are written once and run on many platforms (via the Java Virtual Machine)
- Large standard library and huge ecosystem of tools and frameworks
- Used in education, industry, research, and open-source software

Java is ...

A language

- Syntax and rules
- Types, classes, methods
- What you type in `.java` files

A platform

- JVM + libraries
- Tools (compiler, debugger)
- Runtime services

An ecosystem

- Build tools (Maven, Gradle)
- IDEs (IntelliJ, Eclipse, VS Code)
- Frameworks (Spring, Jakarta EE, ...)
- Community and documentation

A first mental model

Three layers (for now)

1. **You** write Java source code (`.java`)
2. The **compiler** (`javac`) turns it into bytecode (`.class`)
3. The **JVM** executes that bytecode on your machine

You do not need hardware-specific machine code for each OS— the JVM handles that difference.

Hello, Java (preview)

```
1 public class Hello {  
2     public static void main(String[] args) {  
3         System.out.println("Hello, Java!");  
4     }  
5 }
```

- Every application needs a `main` method as an entry point
- `System.out.println` prints a line to the console
- Class name and file name usually match: `Hello.java`

We will unpack this later

For now, treat `public class ...` and `public static void main` as the standard wrapper for a runnable program.

- Later lectures: classes, methods, `static`, access modifiers
- Today: motivation, platform model, and where Java fits

Try this

If you already have a JDK installed, compile and run `Hello.java`. If not, we will set up the toolchain soon.

Quiz 1 — What is Java?

Quiz

Which statement best describes Java?

1. A markup language for web pages (like HTML)
2. A general-purpose, object-oriented language that runs via a JVM
3. A database query language
4. An operating system for servers

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. A general-purpose, object-oriented language that runs via a JVM.

Java programs are written as source, compiled to bytecode, and executed by the Java Virtual Machine—not by the CPU directly as native machine code.

A short history of Java

- **1991–1995:** Started at Sun Microsystems (project “Oak”, then “Java”)
- Goal: write software that could run on many devices and networks
- **1995:** Java 1.0 announced — “Write Once, Run Anywhere”
- Grew with the web (applets era), then enterprise servers
- **2010:** Oracle acquires Sun; Java continues as a major platform
- Today: long-term support (LTS) releases such as Java 17 and Java 21

Design ideas that still matter

From the early design, Java emphasised:

- **Simplicity** relative to C/C++ (no manual memory management for most code)
- **Object orientation** as a primary organising idea
- **Safety**: strong typing, array bounds checks, sandboxing ideas
- **Portability** via bytecode + JVM
- **Productivity**: rich standard library

These choices explain much of Java's classroom and industry success.

Versions: what you need to know

- Java has a versioned language *and* platform
- New features arrive regularly; LTS versions are common in courses and companies
- This course uses **modern Java** (Java 17-friendly examples are fine)
- You rarely need to memorise version numbers early on

Practical tip

Check your version with `java -version`. Prefer a current LTS JDK for coursework.

- Java remains a top language in enterprise software
- Common in banking, telecom, government, e-commerce, and SaaS
- Android (historically and still widely) and JVM-based backends
- Strong hiring signal: types, OOP, testing, and large codebases

Learning Java is not only about one language—it trains habits used in professional development.

Portability: Write Once, Run Anywhere

- Same .class files can run on Windows, macOS, Linux, servers, containers
- Provided a compatible JVM is installed
- Reduces “it works on my machine” friction *at the language/runtime level*

Caveat

Portability is not magic: file paths, encodings, and native libraries still need care.

Java's ecosystem helps you grow beyond toy programs:

- Standard library: collections, I/O, concurrency, networking, . . .
- Build and dependency tools (Maven, Gradle)
- Mature IDEs with refactoring and debugging
- Testing frameworks (e.g. JUnit) used in industry
- Huge amount of tutorials, books, and Stack Overflow answers

Java is a natural vehicle for learning **object-oriented programming**:

- Classes and objects as everyday building blocks
- Encapsulation, inheritance, and interfaces (later in the course)
- Clear separation of “what an object knows” and “what it can do”

These ideas transfer to C#, Kotlin, TypeScript, and many other languages.

Why Java in a first-year course?

- Forces you to be explicit about types and structure
- Errors often appear early (compile-time), which helps learning
- Scales from small exercises to multi-file projects
- Matches many follow-on courses (algorithms, software engineering, databases)

Common mistake

Thinking Java is “only for big companies.” It is also excellent for learning core CS ideas carefully.

Quiz 2 — Why learn Java?

Quiz

True or false?

“Write Once, Run Anywhere” means a Java `.class` file can run on different operating systems *without* rewriting the program, as long as a compatible JVM is available.

1. True
2. False

Pause — answer (clicker / raise hand / neighbour) before the next slide.

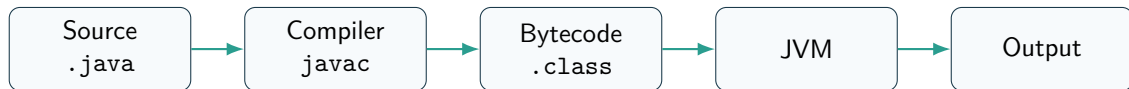
Answer

True. Bytecode is portable across platforms that provide a compatible JVM.

Caveat (from earlier): paths, encodings, and native libraries can still differ— portability is strong at the language/runtime level, not magic for every detail.

From idea to running program

High-level pipeline:



Keep this pipeline in mind whenever something “does not run”: Was it compiled? Is the class name correct? Is a JDK available?

Source code

- Human-readable text in `.java` files
- Must follow Java's syntax rules
- Organised into classes (and later packages)
- Edited in any text editor or IDE

Analogy

Source code is the recipe. The computer cannot “cook” it directly until it is compiled and executed.

Bytecode (high level)

- The compiler produces **bytecode**, not native CPU instructions
- Bytecode is a compact instruction set for the JVM
- Stored typically in `.class` files
- Same bytecode can be shipped to different operating systems

You do not write bytecode by hand in this course.

The Java Virtual Machine (JVM)

The **JVM** is a program that:

- Loads bytecode
- Verifies it (basic safety checks)
- Executes it (often with Just-In-Time compilation for speed)
- Manages memory (garbage collection)

Think of the JVM as a portable “computer” implemented in software.

JDK vs JRE (student view)

- **JDK** (Java Development Kit): compiler + tools + runtime — what *you* install to write programs
- **JRE** (Java Runtime Environment): enough to *run* programs (concept historically important)
- In modern distributions, installing a JDK is the usual student choice

Command-line essentials

```
javac Hello.java    then    java Hello
```


Compile and run (workflow)

```
1 # In a terminal (comments for humans):  
2 javac Hello.java      # creates Hello.class  
3 java Hello            # runs the main method
```

- Do not include .class in the java command
- Class name is case-sensitive

Common mistake

Running `java Hello.java` by habit from other tools. Classic workflow is `javac` then `java ClassName`. (Some newer JDKs can run single-file source directly—nice, but learn the classic steps.)

Why this model helps beginners

- Compile-time errors catch many mistakes early
- Clear separation: editing vs compiling vs running
- Encourages reading error messages carefully
- Mirrors how larger projects are built

Quiz 3 — Compile and run

Quiz

You have a file `Hello.java`. Which pair is the classic workflow?

1. `java Hello.java` then `javac Hello`
2. `javac Hello.java` then `java Hello`
3. `javac Hello` then `java Hello.class`
4. `java Hello` only (no compile step ever)

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. `javac Hello.java` creates `Hello.class`; then `java Hello` runs it.

Do not pass `.class` to the `java` command. Some newer JDKs can run a single source file directly—useful later, but learn the classic two steps first.

Quiz 3b — Bytecode (true/false)

Quiz

True or false?

The Java compiler produces native CPU machine code for your laptop's processor.

1. True
2. False

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

False. The compiler produces **bytecode** for the JVM, not CPU-specific machine code.

The JVM (often with JIT compilation) handles execution on the actual hardware.

Where Java is used

Java appears across many domains:

- **Backend / server** applications and APIs
- **Android** applications (Java/Kotlin on the JVM/Android runtime family)
- **Desktop** tools and cross-platform apps
- **Big data** and streaming (e.g. parts of Hadoop/Kafka ecosystems)
- **Scientific** and financial systems
- **Embedded** and IoT in selected environments

Enterprise and the web

- Many companies run large Java services for years
- Frameworks help structure web apps and microservices
- Strong typing and tooling support large teams
- Reliability and maintainability often matter as much as raw speed

You will not build a full enterprise stack in week one— but the language you learn here is the same foundation.

Related languages on the JVM

The JVM hosts more than Java:

- **Kotlin** — popular for Android and modern JVM apps
- **Scala**, **Clojure**, and others for specialised styles

Learning Java first makes these easier to approach later.

Try this

After class, skim the Oracle or OpenJDK “What is Java?” overview page and note three facts you did not know.

Java vs other first languages (honest comparison)

Aspect	Java	Often compared to
Typing	Static, explicit	Python (dynamic)
Verbosity	More ceremony early	Scripting languages
Feedback	Compile + run	Often interpret/run
OOP	Central	Varies
Industry	Very strong	Depends on domain

None is universally “best”—Java is a

strong teaching and career choice.

Quiz 4 — Where Java is used

Quiz

Which domain is a *typical* place you will meet Java in industry?

1. Backend / server applications and APIs
2. Replacing HTML as the only language for static web pages
3. Writing GPU shader programs instead of GLSL
4. Assembling BIOS firmware for every laptop brand

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

1. Backend / server applications and APIs (also Android/JVM apps, tools, finance, and more).

Java is not “only enterprise,” but large server-side systems are a very common home for it.

We will:

- Start with syntax and small programs you can reason about
- Emphasise reading code, not only writing it
- Introduce OOP concepts carefully, with examples
- Practise debugging and explaining errors
- Build toward structured, multi-class programs

Habits that help

- Type examples yourself—do not only read slides
- Change one thing at a time and observe the result
- Keep programs small until ideas are solid
- Name variables meaningfully (`score`, not `x1`)
- Ask: “What is the state of my program after this line?”

Tools you will use

- A JDK (OpenJDK or another distribution)
- A terminal for `javac` / `java`
- An IDE or editor (course recommendation is IntelliJ IDEA)
- Version control later (e.g. Git) for projects

Setup details belong in a lab session; conceptually you now know *why* the JDK matters.

Key takeaways

1. Java is a portable, object-oriented language with a large ecosystem
2. Career, portability, tools, and OOP foundations motivate learning it
3. Source $\rightarrow$ bytecode $\rightarrow$ JVM is the core execution story
4. Java powers servers, Android, tools, and many large systems
5. This course builds from small, clear programs toward OOP design

What questions do you have about Java so far?

Suggested reading / practice

- Skim an official “Introduction to Java” overview
- Install a JDK and run `java -version`
- Compile and run a Hello program